

SONIQ MASTER AI

FULL BLUEPRINT + CODE + ORIGINAL VOCAL PRESERVATION

V1 — Rule-Based AI + DSP + Vocal Preservation + QC

Spesifikasi pembangunan praktikal: fungsi setiap modul, contoh kod, pipeline DSP, database, API, kredit dan perlindungan suara asal.

1. PRINSIP UTAMA

SONIQ ialah mastering engine, bukan voice changer. Pitch, formant, timing dan identiti suara tidak diubah dalam mode mastering.

```
ORIGINAL SONG
-> MIX ANALYSIS
-> VOCAL PRESERVATION ANALYSIS
-> AI MASTERING DECISION
-> DSP MASTERING
-> VOCAL SAFETY QC
-> FINAL MASTER
```

2. MODUL WEBSITE & KEGUNAAN

Landing Page: penerangan dan CTA. Dashboard: projek, sejarah, kredit dan job. Upload: menerima/validate audio. Analysis: loudness, spectrum, dynamics, stereo, clipping. Vocal Analysis: protection. Mastering Mode: AUTO/CUSTOM/REFERENCE. Style Selector: Natural, Clean, Warm, Punch, Wide, Power, Vintage, Cinematic. Advanced Controls: Bass/Vocal/Clarity/Punch/Width/Warmth/Loudness. Reference: bandingkan reference. A/B: Original vs Master loudness-matched. Report: terangkan perubahan. Credits/Subscription: pembayaran.

3. STRUKTUR FOLDER

```
soniq-master-ai/
├── apps/web/app/
├── apps/api/{routes,services,workers,schemas}
├── mastering-engine/
│   ├── analysis/
│   ├── vocal/
│   ├── ai/
│   ├── dsp/{eq,compressor,multiband,saturation,stereo,limiter}
│   ├── pipeline/
│   └── qc/
├── database/
├── presets/
├── tests/
└── infrastructure/
```

4. UPLOAD API

Pintu masuk audio. Production perlu size limit, MIME verification, malware scan dan private object storage.

```
@app.post("/api/v1/audio/upload")
async def upload_audio(file: UploadFile):
    allowed = {"audio/wav","audio/x-wav","audio/flac",
              "audio/aiff","audio/mpeg"}
    if file.content_type not in allowed:
        raise HTTPException(400, "Unsupported format")
    # Save privately; create audio_files DB record.
    return {"status":"uploaded","filename":file.filename}
```

5. AUDIO ANALYSIS

Menghasilkan feature vector untuk AI. Boleh dianalisis full track dan segmen intro/verse/chorus/drop/outro.

```
def analyze_mix(audio):
    return {
        "lufs": measure_integrated_lufs(audio),
        "true_peak": measure_true_peak(audio),
        "crest_factor": measure_crest_factor(audio),
        "dynamic_range": measure_dynamic_range(audio),
        "spectrum": analyze_spectrum(audio),
        "stereo_width": analyze_stereo(audio),
        "clipping": detect_clipping(audio)
    }
```

6. ORIGINAL VOCAL PRESERVATION

Mengesan kawasan vokal dan risiko sibilance/harshness supaya processing global tidak merosakkan suara penyanyi.

```
def vocal_safety_profile(audio):
    vocal = estimate_vocal_presence(audio)
    return {
        "vocal_presence": vocal.presence_score,
        "main_band": vocal.main_band,
```

```

    "dynamic_range": vocal.dynamic_range,
    "harshness": detect_vocal_harshness(vocal),
    "sibilance": detect_sibilance(vocal)
}

```

7. PERATURAN SUARA ASAL

Suara penyanyi dan karakter vokal mesti kekal. Tiada voice cloning/replacement dalam mastering mode.

DO NOT CHANGE:

- pitch
- formant
- timing
- voice identity
- vocal character

ALLOWED:

- gentle EQ
- dynamic control
- de-essing
- light saturation
- level balancing
- transparent limiting

8. AI DECISION ENGINE

AI memproses masalah yang dikesan, bukan menggunakan fixed EQ preset.

```

def build_mastering_plan(features, genre, style):
    plan = MasterPlan()
    if features["bass_energy"] > 0.80:
        plan.eq.low_shelf = -0.7
    if features["harshness"] > 0.60:
        plan.dynamic_eq.add(frequency=7000, max_cut=-2.0)
    if features["dynamic_range"] < 6:
        plan.compressor.bypass = True
    if features["stereo_width"] > 0.85:
        plan.stereo.amount = 0.0
    if features["vocal_presence"] > 0.70:
        plan.eq.vocal_band_max_change = 1.0
    return plan

```

9. EQ ENGINE

Tonal correction. Vocal protection zone mengehendkan perubahan pada mid/presence.

```

eq.set_band(250, -0.8, 0.9, "bell")
eq.set_band(10000, 0.4, 0.7, "high_shelf")

```

10. DYNAMIC EQ / DE-ESSING

Mengawal sibilance dan harshness hanya apabila muncul.

```

def vocal_high_control():
    return DynamicBand(
        frequency=7200, threshold=-20,
        ratio=2.0, range_db=-2.0)

```

11. COMPRESSOR

Mengawal dynamics secara ringan. Vokal expressive tidak boleh diratakan berlebihan.

```

compressor = {
    "ratio": 1.5, "attack_ms": 30,
    "release_ms": 120, "max_gain_reduction_db": 2.0
}
if mix_is_already_compressed:
    compressor["bypass"] = True

```

12. MULTIBAND

Kawalan dynamics mengikut frequency range; hanya band tidak stabil diaktifkan.

```

bands = [
    ("sub", 20, 80), ("low", 80, 250),
    ("mid", 250, 2000), ("high", 2000, 8000),
    ("air", 8000, 20000)]

```

13. SATURATION

Menambah harmonic density secara halus; bypass jika distortion tinggi.

```
def saturation_amount(features):
    if features["distortion"] > 0.50:
        return 0.0
    return min(0.08, 0.03 + features["warmth_need"]*0.05)
```

14. STEREO IMAGING

Memberi ruang tanpa merusakkan phase, mono compatibility atau pusat vokal.

```
def stereo_policy(features):
    if features["phase_risk"] > 0.60:
        return {"width":0.0,"low_end_width":0.0}
    return {"low_end_width":0.0,"mid_width":0.03,"high_width":0.08}
```

15. LIMITER & TRUE PEAK

Menetapkan loudness sasaran dan melindungi true peak. Nilai perlu divalidasi.

```
def final_limiter(target_lufs=-11.0, true_peak=-1.0):
    return {"target_lufs":target_lufs,
            "true_peak_db":true_peak,
            "oversampling":True}
```

16. MASTERING PIPELINE

Menggabungkan analysis, AI, DSP dan QC.

```
def master_song(audio, settings):
    a = analyze_mix(audio)
    v = vocal_safety_profile(audio)
    plan = build_mastering_plan(**a,**v),
        settings.genre,settings.style)
    x = apply_eq(audio, plan.eq)
    x = apply_dynamic_eq(x, plan.dynamic_eq)
    x = apply_compressor(x, plan.compressor)
    x = apply_multiband(x, plan.multiband)
    x = apply_saturation(x, plan.saturation)
    x = apply_stereo(x, plan.stereo)
    x = apply_limiter(x, plan.limiter)
    qc = run_qc(analyze_mix(x), v)
    return x, qc
```

17. VOCAL SAFETY QC

Final gatekeeper; master boleh ditolak dan dirender semula jika perubahan terlalu agresif.

```
def vocal_qc(original, mastered):
    return {
        "pitch_changed": False,
        "formant_changed": False,
        "clipping": detect_clipping(mastered),
        "excessive_vocal_change":
            detect_excessive_vocal_change(original, mastered),
        "phase_problem": detect_phase(mastered)}
```

18. ITERATIVE RENDERING

Correction pass kedua jika QC gagal.

```
for attempt in range(2):
    master = render(audio, plan)
    report = qc(master)
    if report["pass"]:
        break
    plan = reduce_processing(plan, report["reasons"])
```

19. A/B LOUDNESS MATCHING

Membolehkan penilaian adil terhadap tone/dynamics.

```
def loudness_match(original, mastered):
    delta = measure_lufs(original) - measure_lufs(mastered)
    return apply_gain(mastered, delta)
```

20. MASTERING API + QUEUE

Audio processing berat berjalan pada worker.

```
@app.post("/api/v1/master")
async def create_master(request):
    job = create_job(request)
    reserve_credit(request.user_id, 1)
    enqueue("mastering-worker", job.id)
    return {"job_id":job.id, "status":"queued"}
```

API -> REDIS QUEUE -> ANALYSIS WORKER
-> MASTERING WORKER -> EXPORT WORKER

21. DATABASE

PostgreSQL: users, subscriptions, projects, audio_files, analysis_results, vocal_analysis, mastering_jobs, mastering_parameters, masters, references, credit_transactions.

22. PRESET

Preset ialah starting target; AI masih menyesuaikan parameter.

```
{
  "name":"Vocal Safe Natural",
  "max_eq_move_db":1.5,
  "compressor_gr_db":1.5,
  "saturation":0.03,
  "stereo_high":0.04,
  "true_peak":-1.0,
  "preserve_dynamics":true,
  "protect_vocal":true
}
```

23. CREDIT & SUBSCRIPTION

Preview 0; Standard Master 1; Reference Master 2; Album Song 1. Job gagal → refund. Pelan contoh: Free, Creator, Pro, Studio.

```
if job_failed:
    refund_credit(user_id, credits_used)
```

24. SECURITY & PRIVACY

Authentication, signed URLs, rate limiting, validation, malware scan, isolated workers dan private storage. Audio private by default.

25. TECH STACK

Frontend: Next.js/React/TypeScript/Tailwind. Backend: Python/FastAPI/Pydantic. Database: PostgreSQL. Queue: Redis. Audio: FFmpeg + custom DSP. AI: PyTorch untuk fasa ML. Storage: S3-compatible. Deploy: Docker/Linux/Nginx.

26. MVP ROADMAP

Phase 1: Upload → Analysis → Vocal Protection → Rule AI → EQ → Compressor → Limiter → QC → A/B → Export.
Phase 2: Dynamic EQ, Multiband, Saturation, Stereo, Reference, Multiple Versions. Phase 3: Credits, Subscription, Payment, Album/Batch. Phase 4: ML, Personalization, API, Studio.

27. TESTING LAGU SEBENAR

Uji Balada Melayu yang menonjolkan vokal dan Techno seperti GEGAR RPM yang menuntut kick, bass, transient dan loudness. Simpan Original, Master, Analysis dan QC report.

28. GOLDEN TEST

Setiap update: render golden tracks → ukur LUFS → True Peak → dynamics → spectrum → stereo/phase → vocal safety → human listening test → approve/rollback.

For every engine update:
render golden tracks
measure LUFS / True Peak / dynamics
compare spectrum and stereo
run vocal safety
human listening test
approve or rollback

29. FINAL USER FLOW

UPLOAD → ANALYSE → CHOOSE STYLE/REFERENCE → AI DECISION → MASTER → VOCAL SAFETY QC → A/B
LOUDNESS MATCH → DOWNLOAD WAV/MP3.

```
graph TD
  USER --> UPLOAD_SONG[UPLOAD SONG]
  UPLOAD_SONG --> ANALYSE
  ANALYSE --> AI_VOCAL_PROTECTION[AI + VOCAL PROTECTION]
  AI_VOCAL_PROTECTION --> DSP_MASTERING[DSP MASTERING]
  DSP_MASTERING --> QC
  QC --> AB[A/B]
  AB --> DOWNLOAD[DOWNLOAD]
```

30. PRINSIP AKHIR

SONIQ ialah adaptive mastering system. Genre ialah konteks; style ialah target character; AI membuat keputusan; DSP memproses; QC memastikan keselamatan; vokal asal dilindungi. Kod ini ialah skeleton/pseudocode dan perlu implementation DSP production, regression tests serta listening validation sebelum deployment komersial.